

LESSON 8

Errors in Dataset Similarity in Datasets and Images



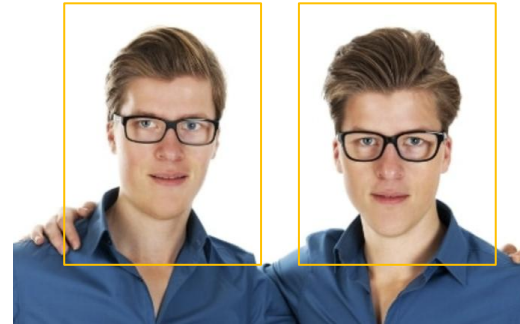
Outline



$y = \text{«cat»}$

- **Labelling errors**

- Supervisor errors
- Changes in time
- Checks



- **Similarity and Distance**

- The general framework of pattern recognition
- Metrics in Datasets
 - Euclidean, Manhattan, Cosine, Chebyshev, Minkowski
- Metrics Images
 - Histograms, SIFT, SURF
 - aHash, dHash, pHash

Focus point

What are we doing?

Dataset Design and Creation

Lesson 6 – *Data Gathering, Preprocessing, and Harmonization*

Lesson 7 – *Number of Parameters and Data Leakage*

Lesson 8 – *Errors in Datasets and Similarity in Data and Images*

Lesson 9 – *Similarity and Coding for Image-Based Retrieval Systems*

From raw data to intelligent dataset design

In this sequence of lessons, we moved from acquiring and harmonizing data (Lesson 6), to understanding how the number of parameters must be balanced with available data (Lesson 7). We then studied how to identify and correct errors in labeling, remove duplicates and select the most useful samples, especially in images (Lesson 8).

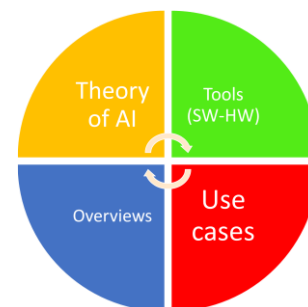
Finally, in Lesson 9, we focus on images and built the basis for designing **image retrieval systems** to create our own intelligent datasets.



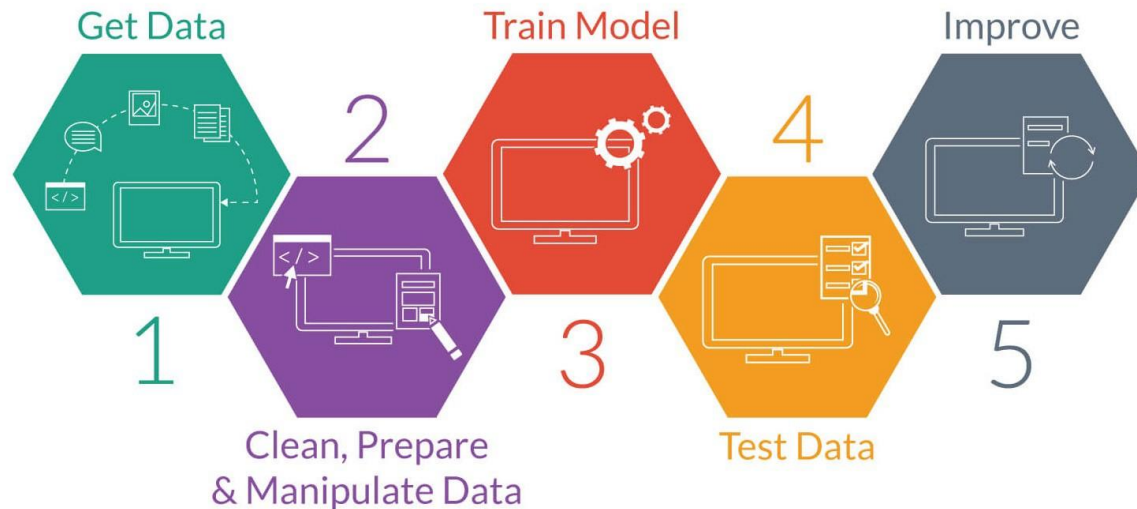
THEORY

Supervisor errors and Labelling errors

Typical errors and their consequences



Step 2 of the ML workflow



Step2 is too often underestimated. If poorly performed, it generates relevant loss of performances

Labels in databases for AI

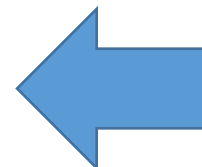
Labels are crucial because they serve as the foundation for training and evaluating machine learning models, particularly in supervised learning

- **Ground truth:** the labels that the AI model should learn to predict or approximate

$Y = \text{FUNC}(X)$ that's it!!!

- For every dataset in machine learning or toolbox, is all about to create X and Y

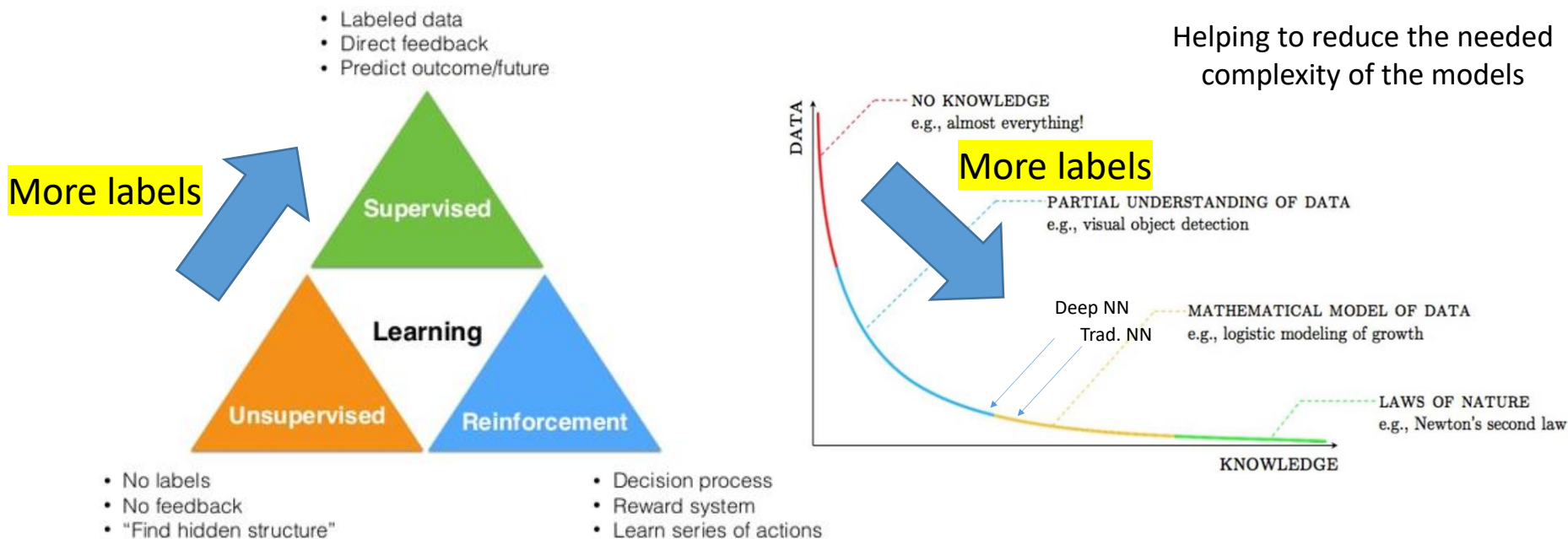
• X	Sample, sample,									
	Features	5.1								
		3.5								
		1.4								
		0.2								
Class ID, or value to be learnt										
• Y		1	0	3	1					



Coded labels
1 = «car», 2 = «truck», ...

Labels in databases for AI

- **Model training:** key in semi and supervised learning



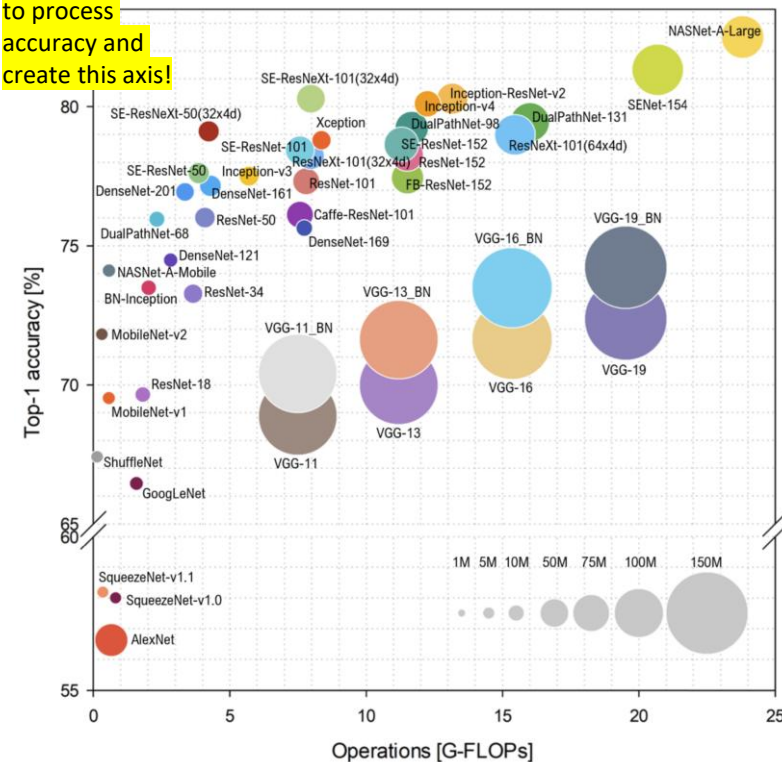
Labels in databases for AI (2)

- **Evaluation/Validation:** to assess the performance of a **single** trained AI model
 - Differently from unsupervised learning where it is difficult to evaluate if the output is correct or not, in supervised learning an accurate assessment of the accuracy is possible based on the labels
(in the next lessons)
- **Benchmarking/Ranking:** **different** models

From labels you can process figures of merits (e.g., accuracy). Labeled datasets are commonly used as benchmarks for comparing the performance of different AI models, algorithms, or approaches

Accuracy VS Operations plot

You need labels to process accuracy and create this axis!



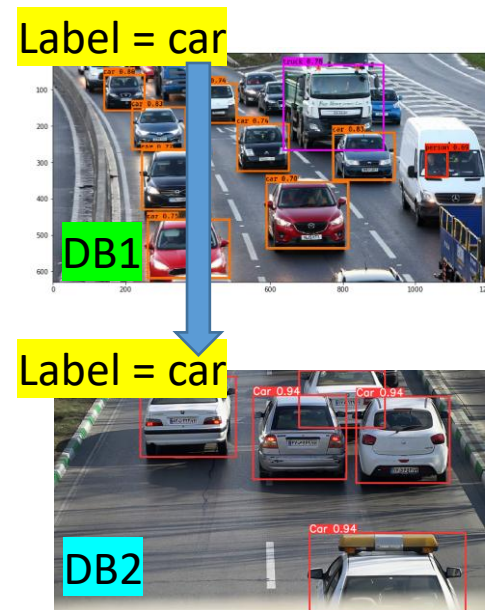
Labels in databases for AI (3)

- **Model interpretability:** labels can also help explain the AI model's decisions, making it more transparent and interpretable.
 - Supervised dataset helps explainability



$y = \text{«cat»}$

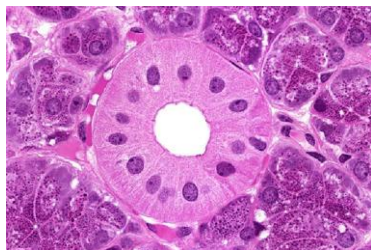
- **Transfer learning:** Labeled data from one domain or task can be used to pre-train models for other related tasks or domains saving time and resources, specially when labeled data is scarce for the target domain
 - Adding specific images according the labels which are present



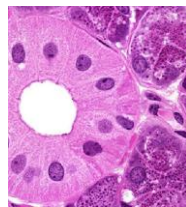
Label Errors in Datasets

Some examples of labeling errors
are from the previous lessons

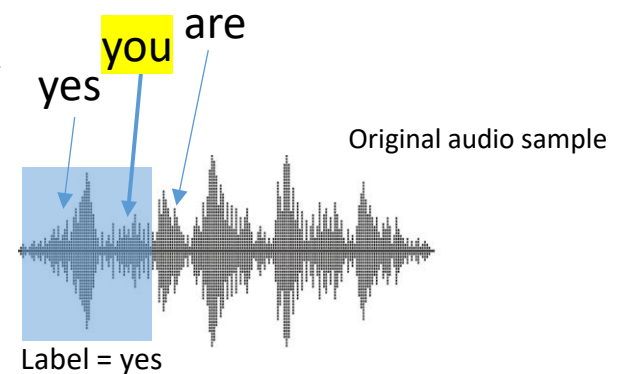
- Sample is correct, but the label is wrong
- The sample is misleading or incomplete (Data Leakage)



Original image sample



The selected cell is
poorly sliced



The selected frame for the
training of the word «YES» is too
long and contains also «YOU»

Label Errors in Datasets

Human supervisors are the typical sources of labels for our datasets

- Direct supervisor errors



$y = \text{«cat»}$

- Example: Medical Diagnosis
 - Inputs = XRAY, blood exams
 - Output = diagnosis



$y = \text{«healthy»}$

Label Errors in Datasets (2)

A Diagnosis can change in time
→ labels can be not updated



X



$y = \text{«b. penumonia»}$

This will limit
the capability
of the AI
model to
perform **early
detection** of
the problem

Second Xray image
with a better machine



$y = \text{«covid-19»}$

Label Errors in Datasets

- Data interchange/automatic conversion errors

```
Patient XML data structure
<doctor authorized code = %code>
<data = %data>
<robot number = %robot>
<person = %name>
  <General>
    <hemoglobin = %>
    <oxygen = %>
    <temperature = %>
    <pressure = %>
    <hear rate = %>
  </General>
  <Bio-sensor>
    <sensor 1> data </sensor1>
    <sensor2> data </sensor2>
    <sensor3> data </sensor3>
  </Bio-sensor>
</person>
<real-time data>
  <sensor no = %no>
    Data_stream....
  </sensor>
</realtime data>
```

Automatic
label creation

Automatic
format
conversion

If $(25.0 < \text{BMI} < 30)$
 $y = \text{«overweight»}$

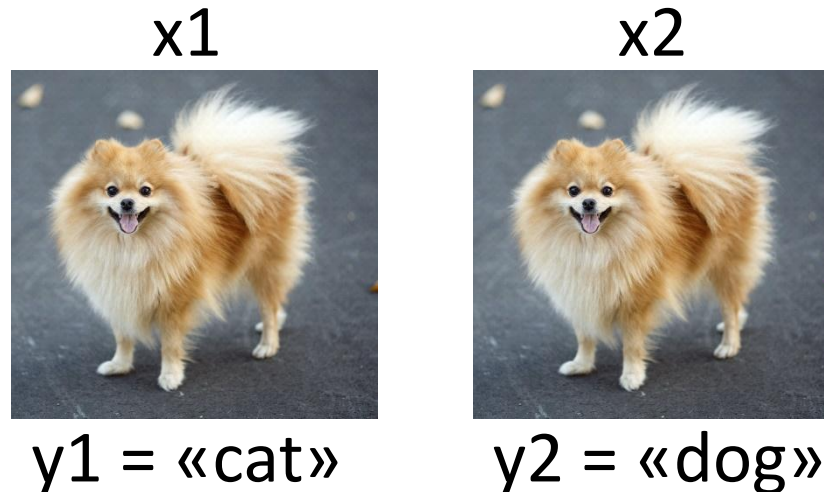
$y = \text{«fat»}$



Jury Chechi @2022 (50+yo)
(Olympic gold medal in Rings 1996)

Basic checks for labelling errors

- Same input vectors (duplications)
 - With **same labels**
 - no training problems, but waste of storage, memory, processing time
 - With **opposite labels** (training problems!)



Basic checks for labelling errors (2)

- What to check in the training and validation database even it is **not easy with large DB** (Dbsize>1M images)
- A simple one to N comparison $\rightarrow \text{iter.} = N*(N-1)$

 x_1  $y_1 = \text{«dog»}$

...

 x_i  $y_i = \text{«cat»}$ x_j  $y_j = \text{«dog»}$

...

 x_N  $y_N = \text{«dog»}$

Basic checks for labelling errors (3)

- Example: the *Serengeti Dataset* (Public)
 - **Unlabeled**: 3.2 million images corresponding to 1.2 million capture events (seq. of images; tot 7.1M images)
 - **Unsupervised is not bad** → Good for tuning, feature extraction, further study
 - **Labelled DB Test set**: volunteer-labeled test set of 17400 capture events.
- A basic check → $\text{iter} = N \cdot (N - 1) = \mathbf{285M}$ comparisons



A very simple procedure

(just a very basic step to start the analysis)

STEP 1) Load x_i and x_j

STEP 2) Check for duplication

STEP 3) Labels are different?

YES // bad case, choose your option

→ OPTION1: Remove the sample

→ OPTION2: Ask for assistance with a supervisor
(second reading)

→ Change the label and merge the sample or reject the sample

NO // good

→ reject one sample

→ manage indexes i, j

→ return to STEP 1)

Which option?

- Extralarge dataset?
- Expensive data?
-

Basic checks for duplications: hash functions

If you are dealing with large data/vectors a simple comparison

`if( x_i == x_j )`

requires **element2element** or **pixel2pixel**
comparison → time consuming
(even if perfectly parallelizable → good for CUDAs)

- Using a **standard** file hash (if already available)
 - Classical MD5 and SHA-1 algorithms
- Create **image-hash** information offline
 - Specific hash functions for images are available

Py Code: hash

```
# MD5 and SHA1 hashes for image comparison
import hashlib

def compute_hashes(image_path):
    with open(image_path, 'rb') as f:
        data = f.read()
        md5_hash = hashlib.md5(data).hexdigest()
        sha1_hash = hashlib.sha1(data).hexdigest()
    return md5_hash, sha1_hash

# Example usage
img1_md5, img1_sha1 = compute_hashes("image1.jpg")
img2_md5, img2_sha1 = compute_hashes("image2.jpg")

print("MD5 Equal:", img1_md5 == img2_md5)
print("SHA1 Equal:", img1_sha1 == img2_sha1)
```

For better performance, **first compute and store the hashes of all images**, then perform the comparisons in a **separate, optimized loop**



Image1.jpg



Image2.jpg

Changing **even 1 pixel** → results in a completely different hash

Modifying **EXIF metadata** (e.g., location, timestamp) → different hash, even if the image looks the same

This method **does not check visual similarity**, it only checks **exact binary identity**

Even if the number of bits of MD5 and SHA1 is different they are ok for our purpose (**MD5 is faster**)

Algorithm	Hash Length (bits)	Hexadecimal Length (characters)
MD5	128 bits	32 hex characters
SHA-1	160 bits	40 hex characters

$2^{128} = 340, 282, 366, 920, 938, 463, 463, 374, 607, 431, 768, 211, 456$

CODE

```
# MD5 and SHA1 hashes for image comparison
import hashlib

def compute_hashes(image_path):
    with open(image_path, 'rb') as f:
        data = f.read()
        md5_hash = hashlib.md5(data).hexdigest()
        sha1_hash = hashlib.sha1(data).hexdigest()
    return md5_hash, sha1_hash

# Example usage
img1_md5, img1_sha1 = compute_hashes("image1.jpg")
img2_md5, img2_sha1 = compute_hashes("image2.jpg")

print("MD5 Equal:", img1_md5 == img2_md5)
print("SHA1 Equal:", img1_sha1 == img2_sha1)
```

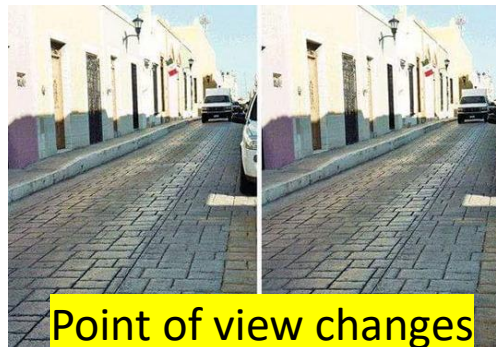
Basic checks for duplications/similarity (3)

Even if you are dealing with images or vectors (or unstructured data) is **NOT** just about

`if( x_i == x_j )`

but something like

`if ( similarity(x_i , x_j) > fixed_threshold )`

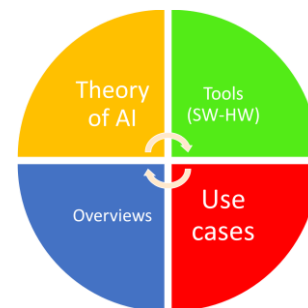
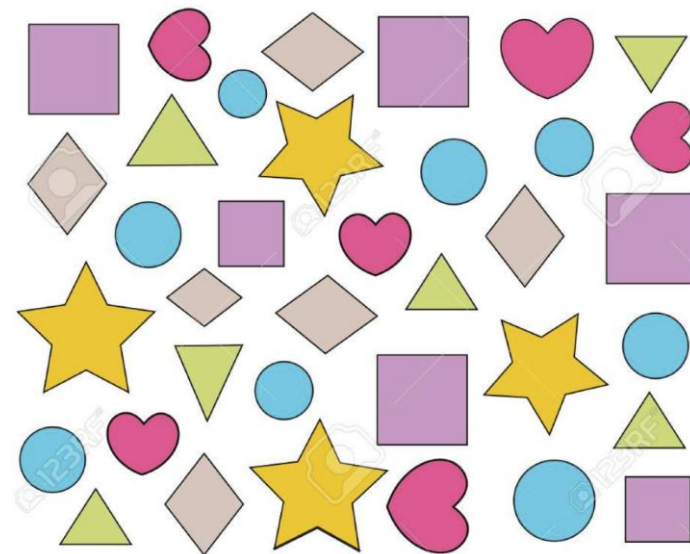




THEORY

Similarity in datasets

What is relevant what is useless?



Tuning the similarity metrics: An example

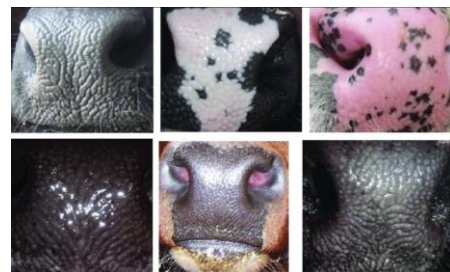
Bottom line:

the similarity metric to be chosen depends on the type of application

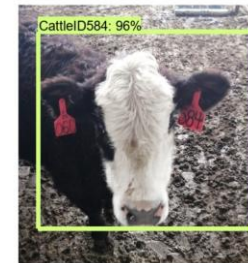
- Caw face recognition
 - Startups are using facial recognition software to increase the productivity of dairy cows.
 - Tracking activity
 - Automatic Food delivery
 - Drug delivery
- Main modules
 - 1 NN to segment the face
 - 1 NN to identify the caw



Features: body spots



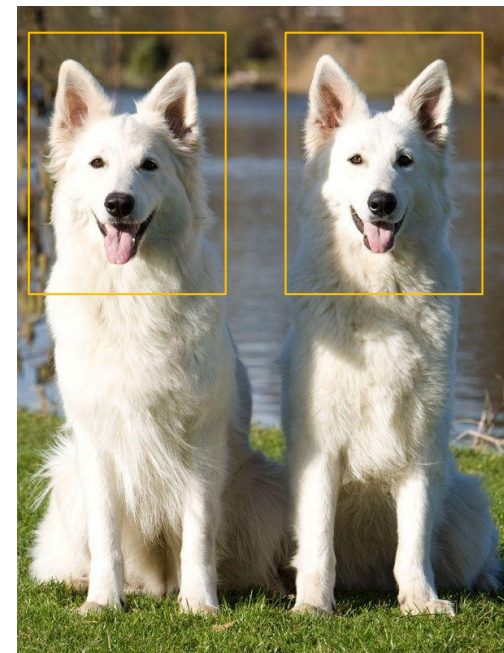
Features:
muzzle spots



Why performing checks for similarity



- Why that?
 - Too similar data/images are providing little more information
 - Make the dataset more complex to be handled
 - Storing data
 - Loading data
 - Training models,
 - Etc.
- The similarity metrics must be tuned according to your application



Two very similar dogs. Too similar to teach the models to identify the breed of the dog, but necessary to identify the single dog

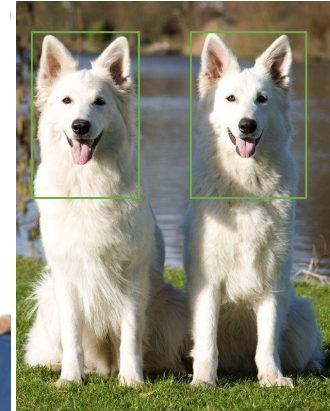
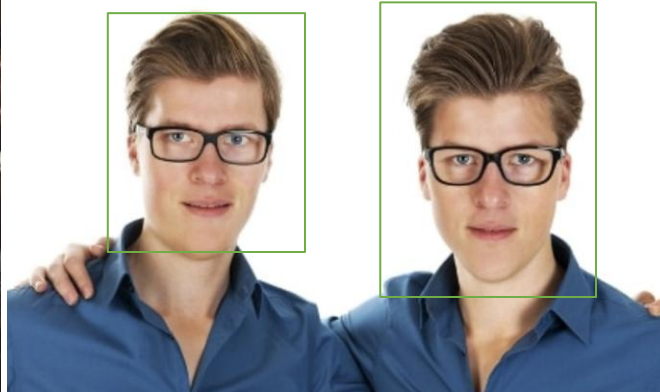
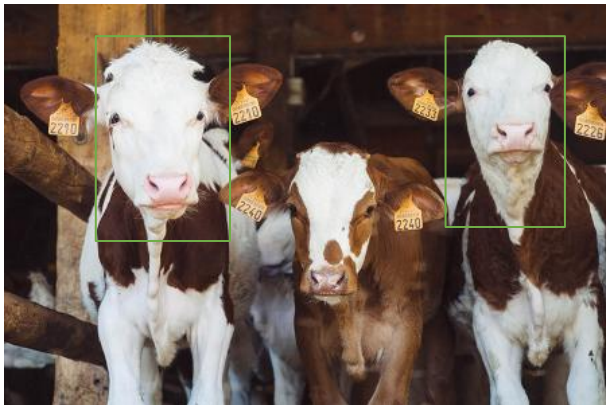
Tuning the Similarity Level

Examples

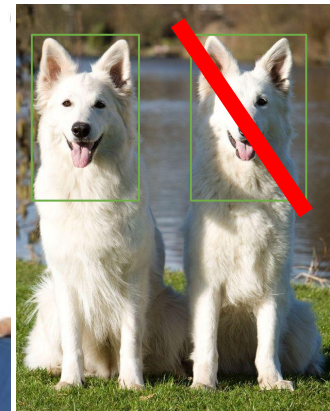
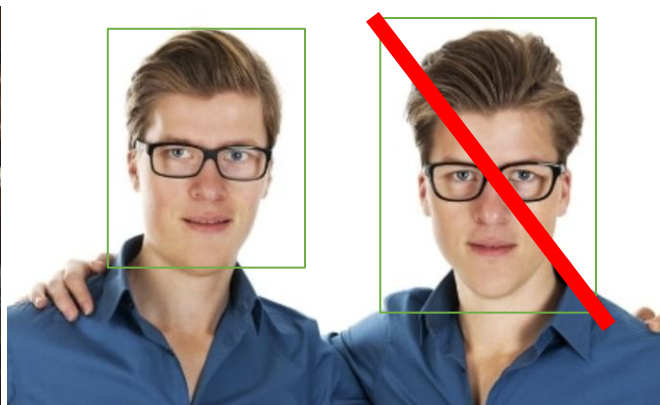
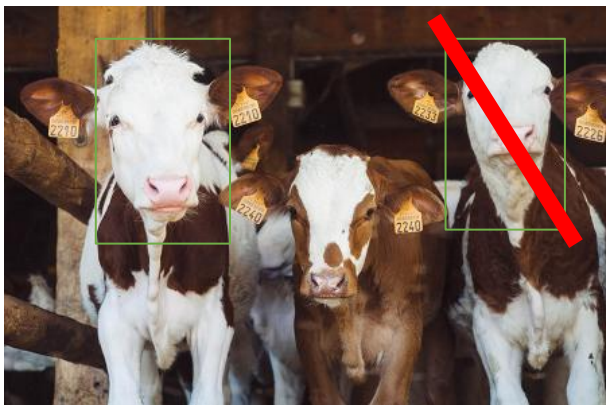
The similarity metrics must be tuned according to your application

... if (**similarity**(x_i , x_j) > fixed_threshold)

For identification task these are very good samples!



For face/snout detection are redundant, better to add more other images



Similarity and data augmentation

Too much **Similarity** → waste of space and time

But...

Data Augmentation → improve generalization

D.A. will be discussed in the deep learning section of the course

Model generalization in ML is how good the model is at learning from the given data and applying the learnt information elsewhere



y=«hotdog»



y=«hotdog»



y=«hotdog»



y=«hotdog»

Similarity in datasets

05nbZ1xxVNWuTcGwLbp7CN	01/24/18	178 NAV	Myself	205,08
07W9fL0ee9LsqD9c3kNrdo	01/28/18	23 Migos	Notice Me	606,51
07W9fL0ee9LsqD9c3kNrdo	01/27/18	16 Migos	Notice Me	763,37
07W9fL0ee9LsqD9c3kNrdo	01/26/18	17 Migos	Notice Me	940,97
09ISlsmFySgyp0piQdqAc	01/31/18	5 Zedd	The Middle	1,028,2
09ISlsmFySgyp0piQdqAc	01/30/18	6 Zedd	The Middle	1,009,0
09ISlsmFySgyp0piQdqAc	01/29/18	7 Zedd	The Middle	900,54
09ISlsmFySgyp0piQdqAc	01/28/18	26 Zedd	The Middle	592,89
09ISlsmFySgyp0piQdqAc	01/27/18	32 Zedd	The Middle	628,88
09ISlsmFySgyp0piQdqAc	01/26/18	36 Zedd	The Middle	678,52
09ISlsmFySgyp0piQdqAc	01/25/18	104 Zedd	The Middle	280,72
09ts3GnlCqYEUSPkQCpJK3	01/31/18	32 Justin Timberlake	Say Something	606,39
09ts3GnlCqYEUSPkQCpJK3	01/30/18	35 Justin Timberlake	Say Something	610,16
09ts3GnlCqYEUSPkQCpJK3	01/29/18	33 Justin Timberlake	Say Something	587,69
09ts3GnlCqYEUSPkQCpJK3	01/28/18	39 Justin Timberlake	Say Something	528,25
09ts3GnlCqYEUSPkQCpJK3	01/27/18	30 Justin Timberlake	Say Something	639,29
09ts3GnlCqYEUSPkQCpJK3	01/26/18	23 Justin Timberlake	Say Something	855,12

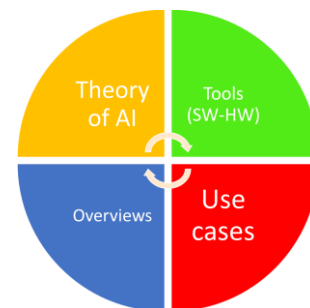
- In the following we focus on images, but a similar approach can be used in general data
- Unstructured data
 - It's better to **extract features** to use structured data techniques
- Structured data → features vectors → metrics
 - Euclidean norm or Manhattan Distance
 - Mahalanobis Distance
 - *Pearson Correlation Coefficient*
 - Complexity, Coherence, Structure, Entropy
 - ...



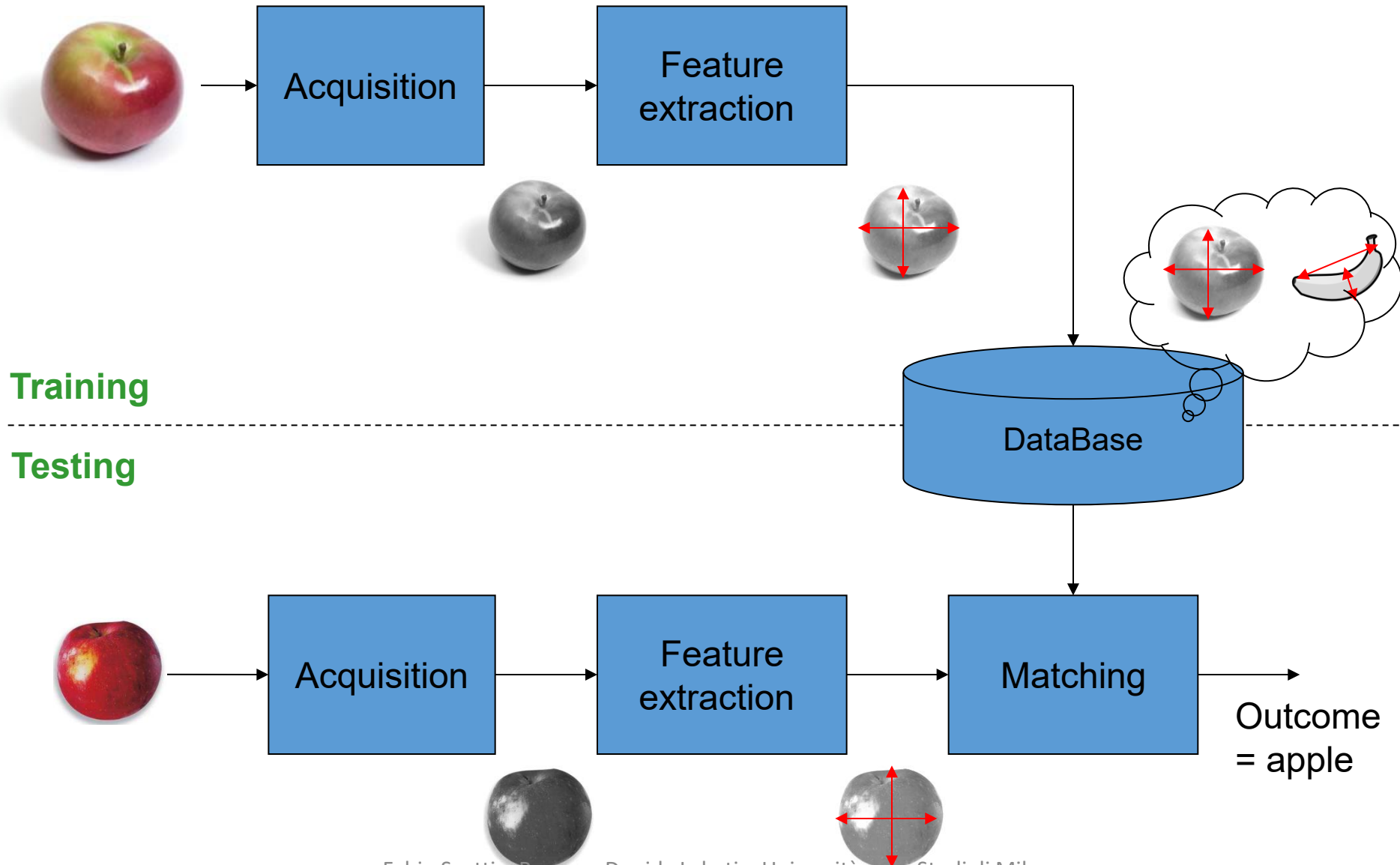
THEORY

Similarity & Pattern Rec.

The basis of most Machine learning methods

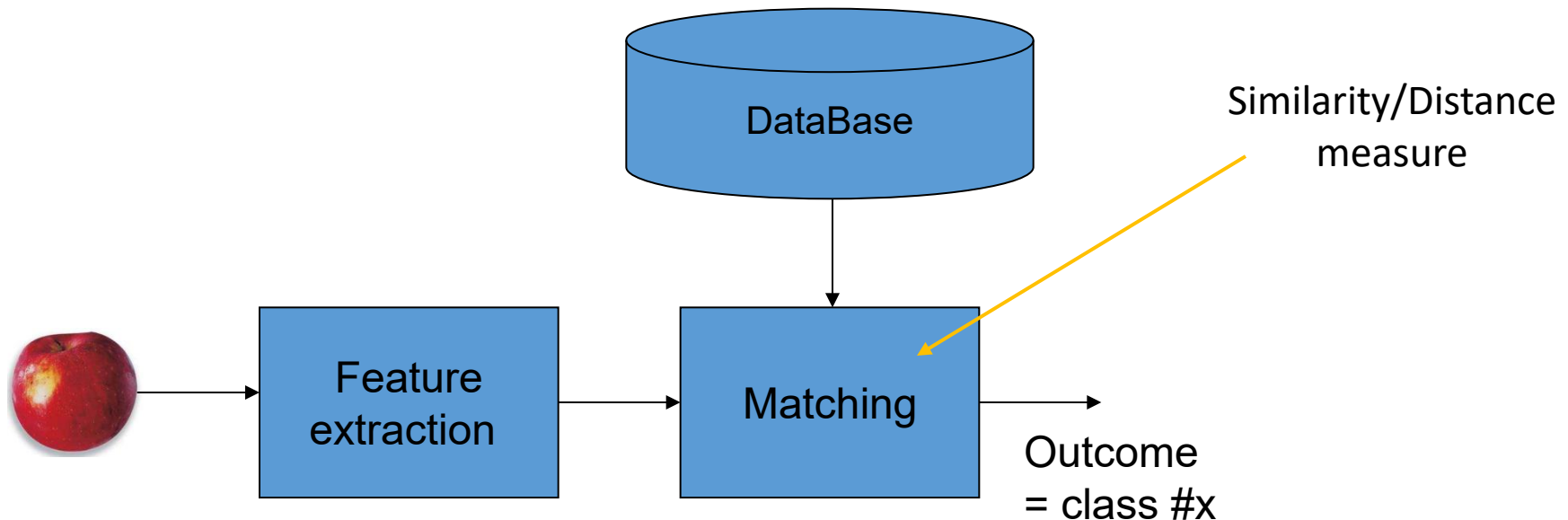


Pattern Recognition Systems



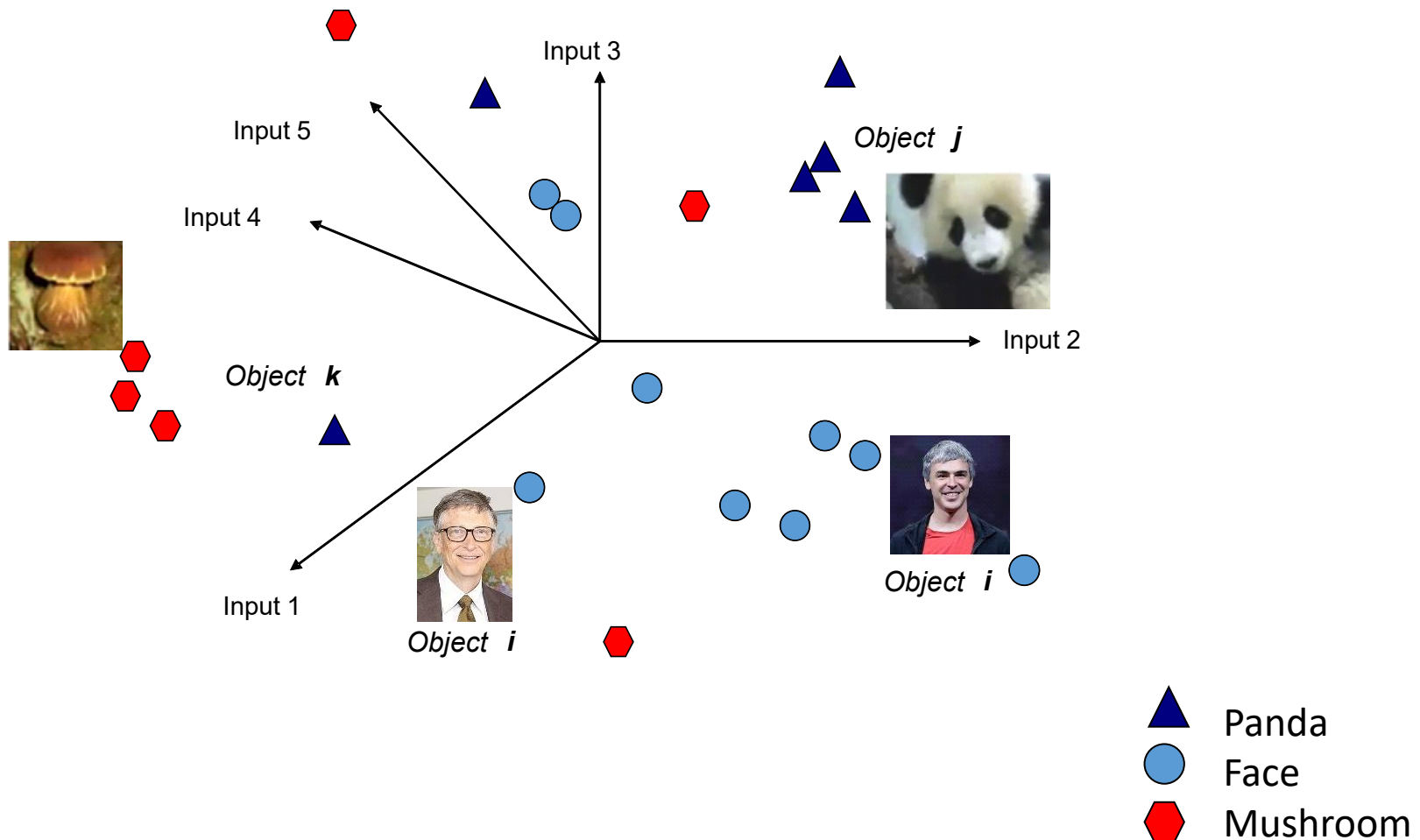
Similarity is the basis of pattern recognition

- Not just to clean our dataset...

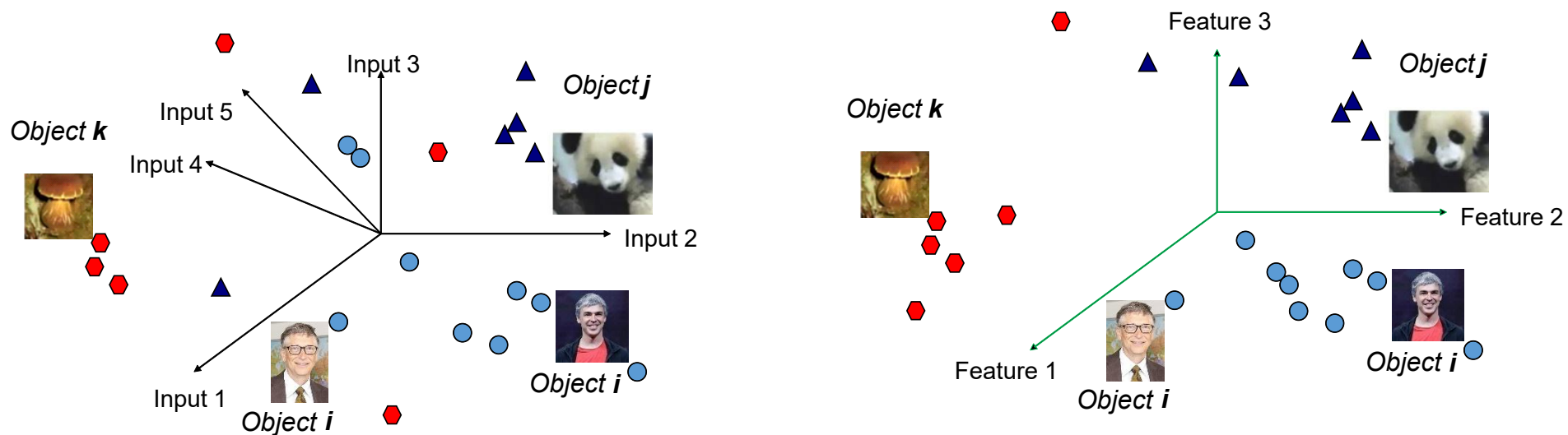
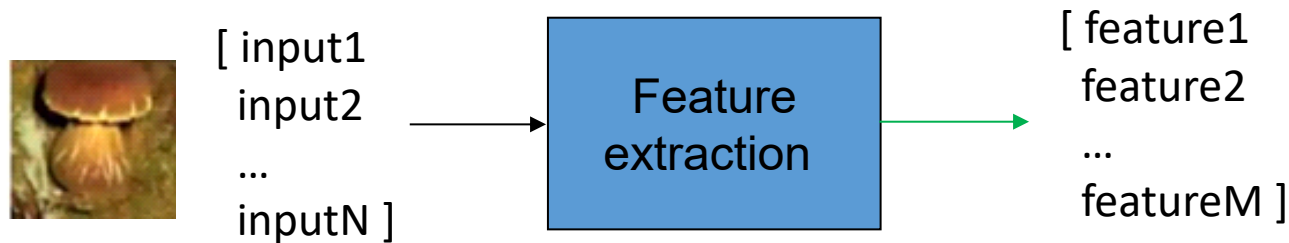


Remember the Input space!

Extracting features (a vector) from structured/unstructured data will allow you to think under a **common framework in ML!**



Input space $\rightarrow$ Feature space

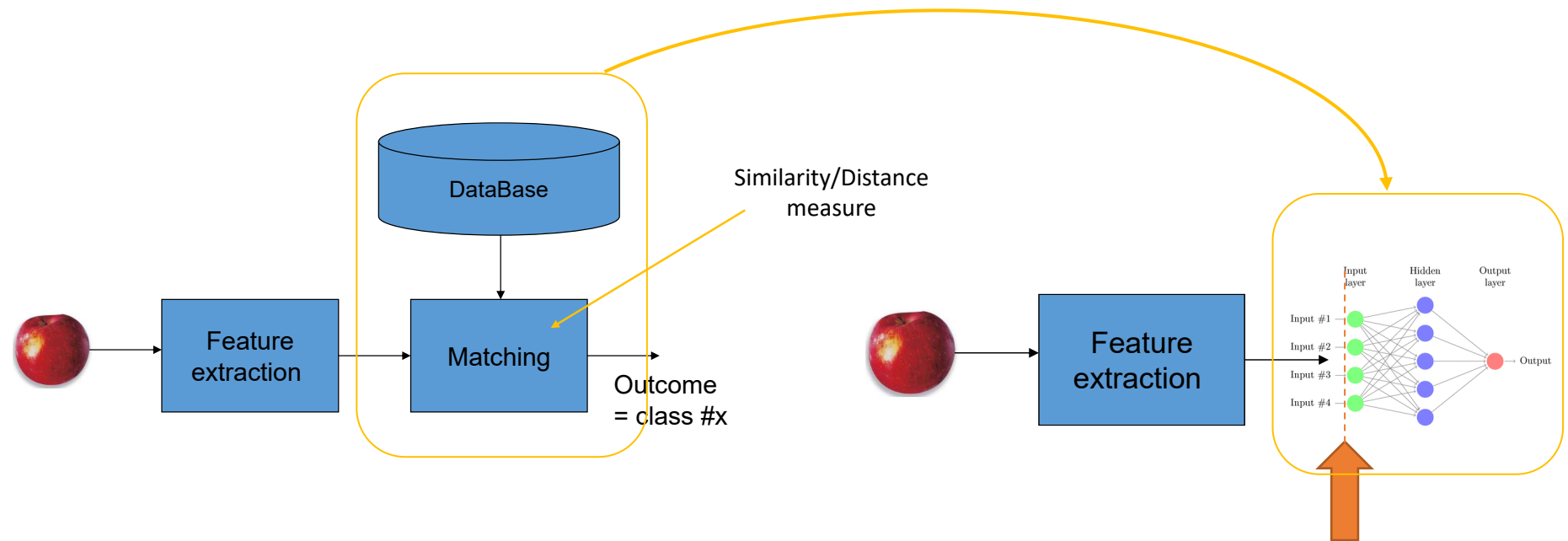


- ▲ Panda
- Face
- Mushroom

Feature Engineering:

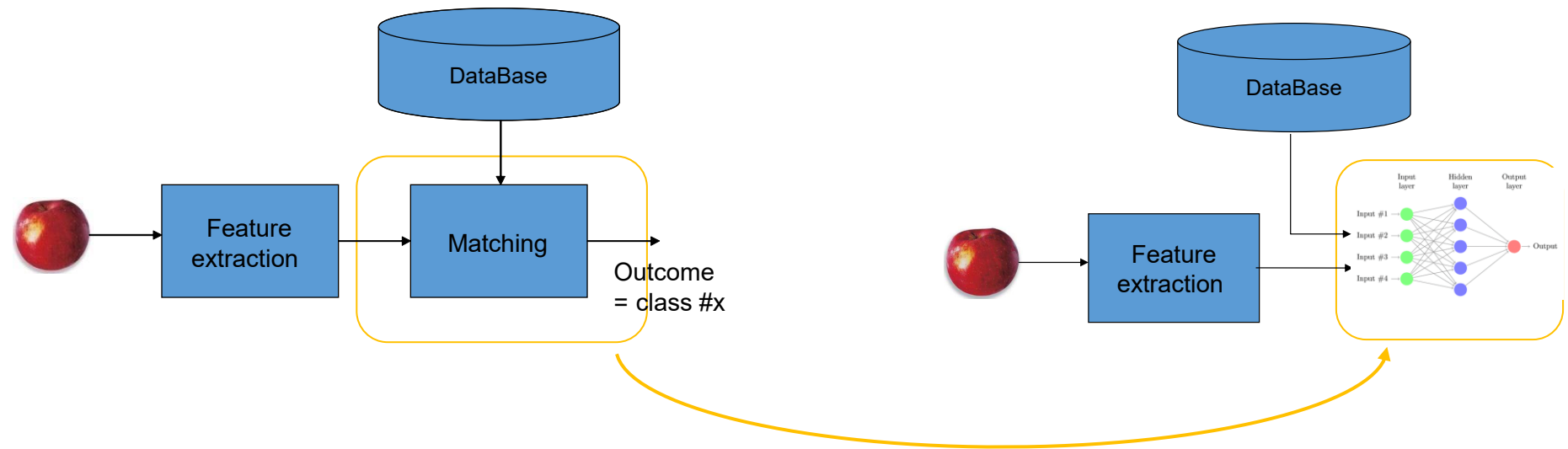
A good design $\rightarrow$ less dimensions and better clusters

What is doing a NN during classification...



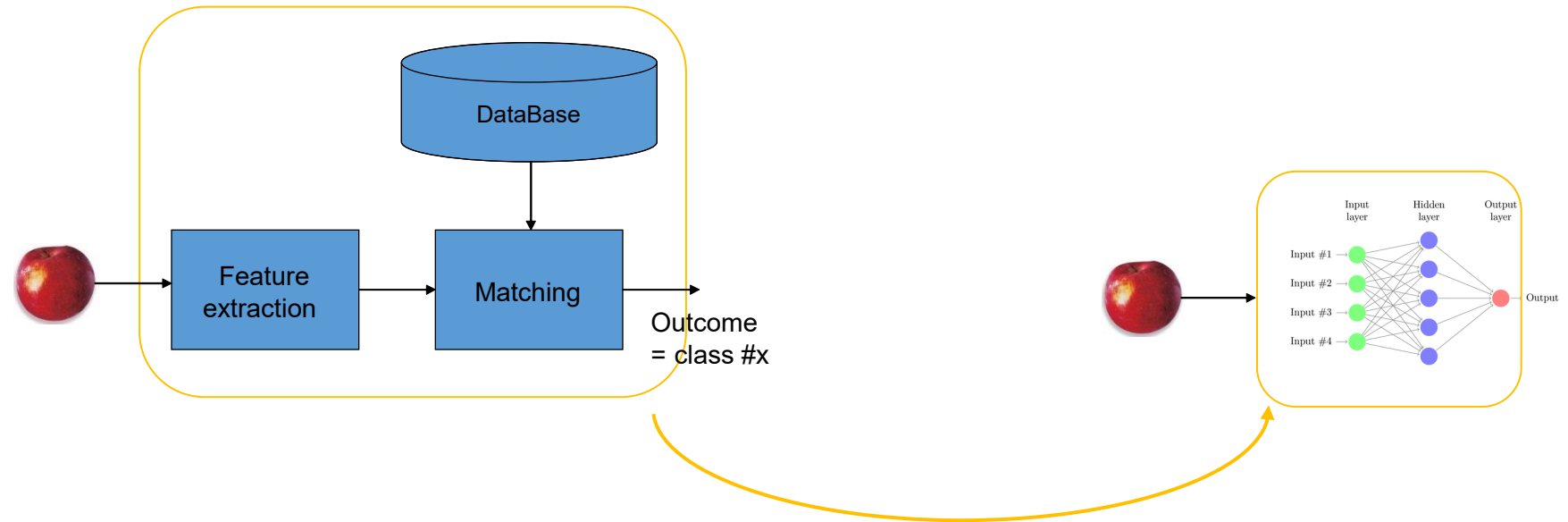
The «storing» and matching function will be mapped by the NN
(complex!)

NN as ... similarity function



Only the matching function will be mapped by the NN
(an easier task to solve)

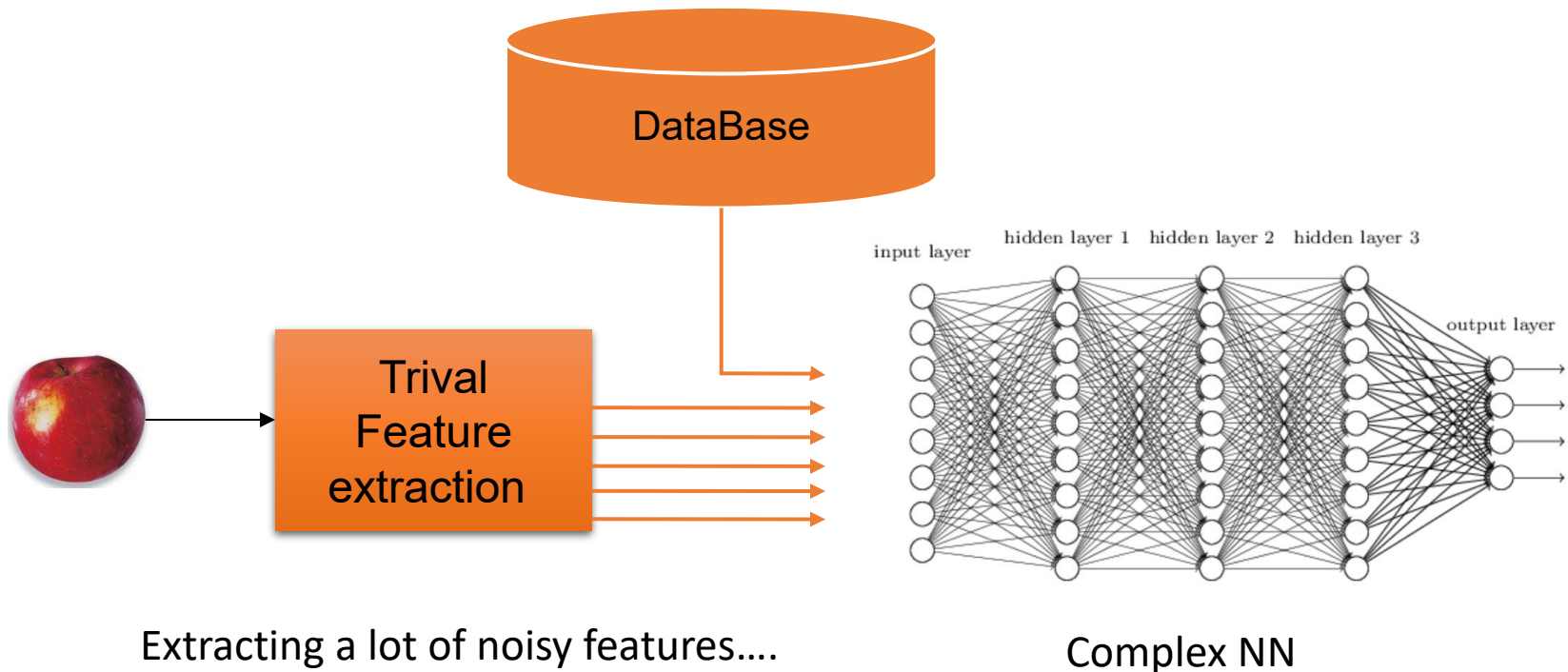
Deep NN



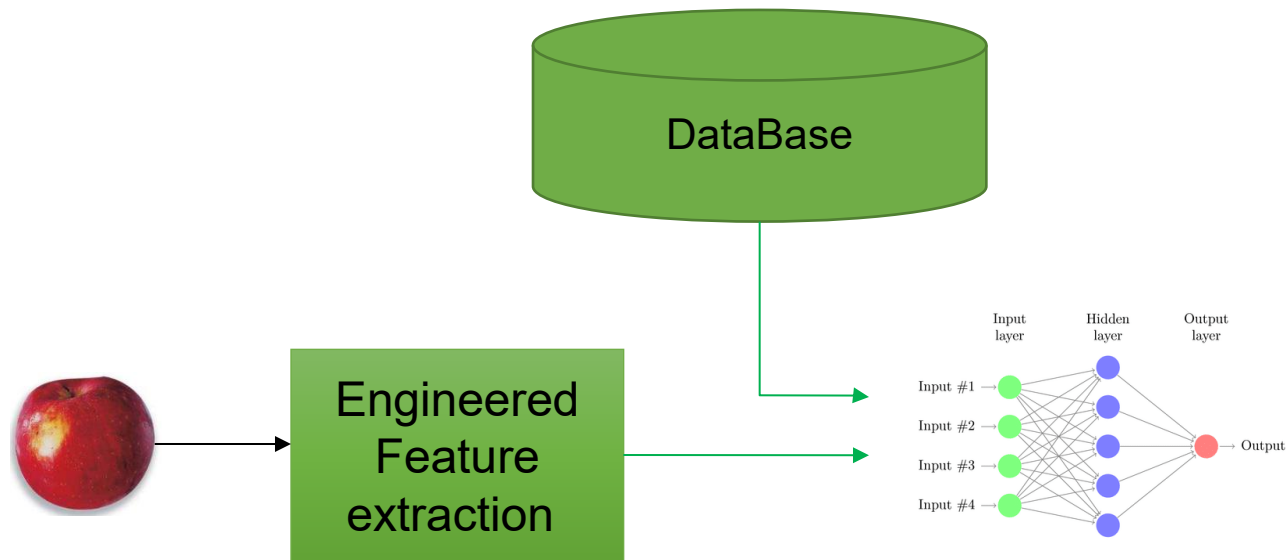
All steps are inside the deep learning model

Trivial feature extraction VS Feature Engineering

Note:
the DB must contain the
same type of features
of the feature extractor



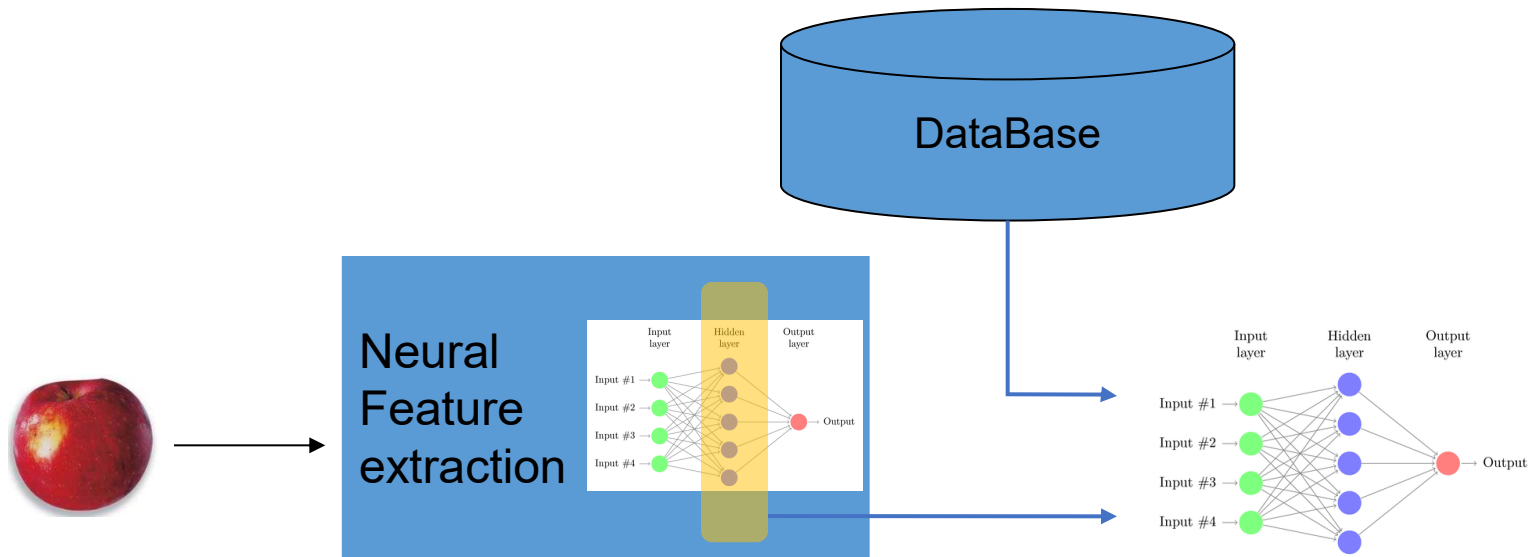
Trivial feature extraction VS Feature engineering



Only few of powerful features....

Simpler NN

Neural feature extraction

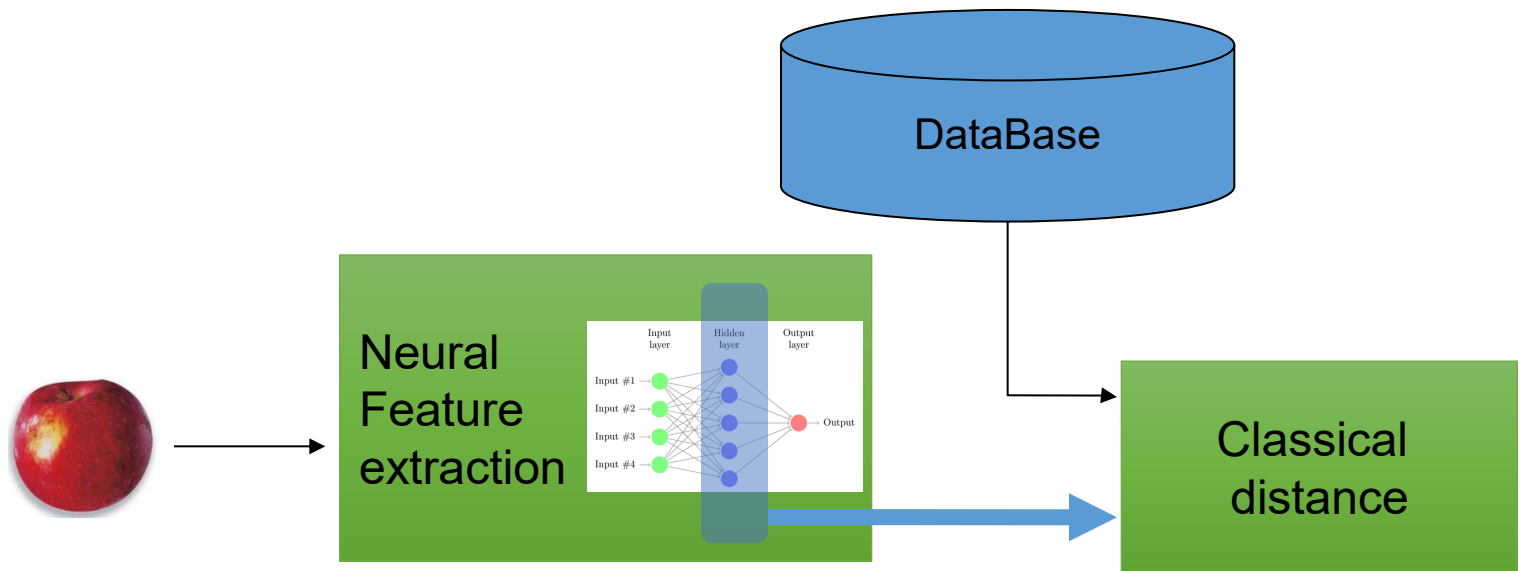


Using NN to extract features
From the hidden layers

Simpler NN

Investing good computation in the first step → simple decisions

... and
improving
explainability



Using NN to extract features
From the hidden layers

E.g., Euclidean distance

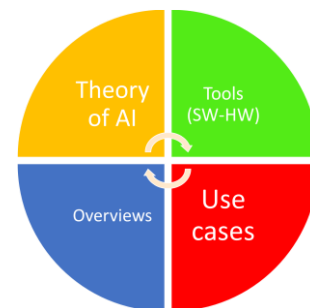
This topic will be discussed later in the course



THEORY

Basic Similarity and Distance in ML

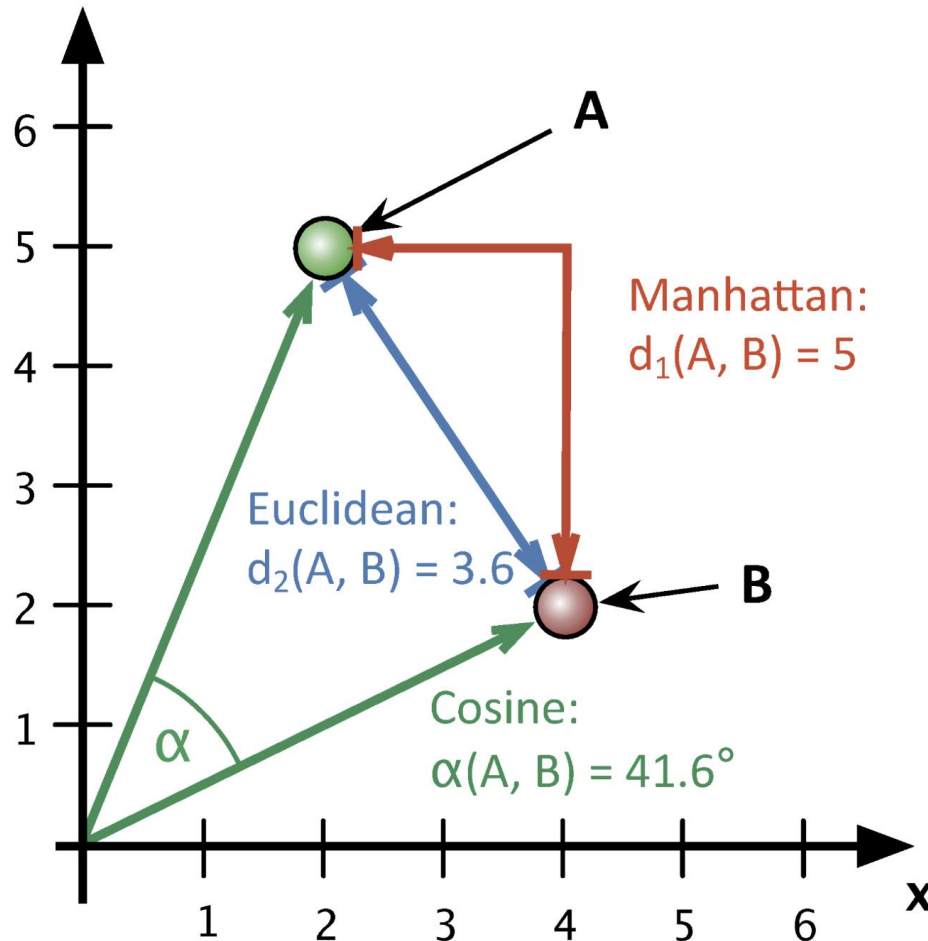
Comparing points in the feature space



Similarity $\leftrightarrow$ Distance

- We think about **similarity**, but often the metrics is the **distance**
- In some sense **is the inverse of distance metrics**
- Main points of the transformation
 - Distance $\rightarrow$ zero so Similarity $\rightarrow$ inf.
Distance $\rightarrow$ inf. so Similarity $\rightarrow$ 0
- The proper conversion depends on the applications and the mathematical properties you would like to have.
 - Example: Similarity = $1 / (1 + \text{distance})$
// $1 + \dots \text{distance} \rightarrow 0$
- Often a real conversion is not necessary, **just use the distance** and find the proper thresholds

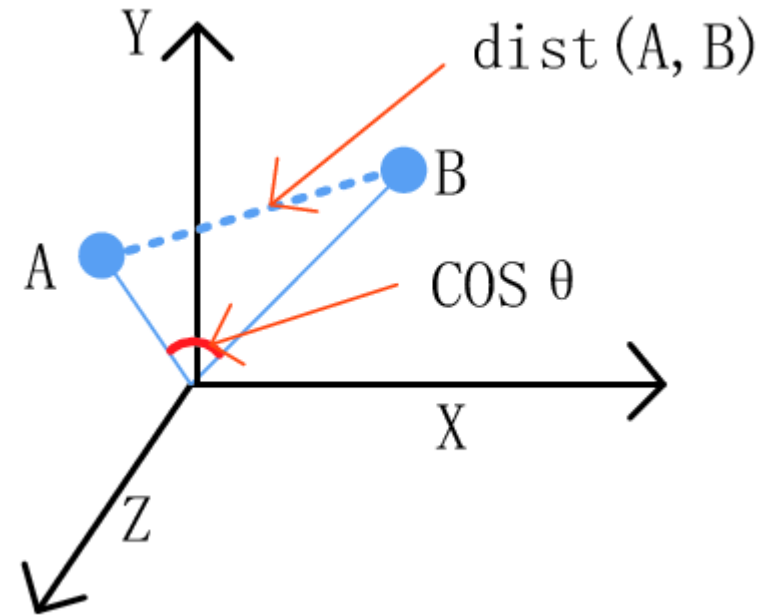
Basic metrics in data similarity $\rightarrow$ distance



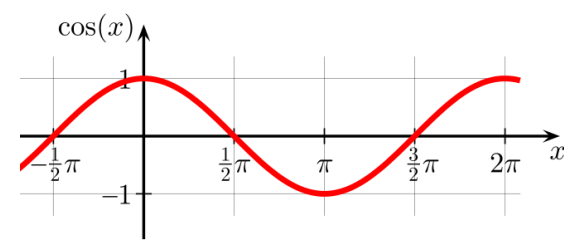
- Euclidean
- Manhattan
- Cosine

Cosine metrics

$$\mathbf{A} \cdot \mathbf{B} = \|\mathbf{A}\| \|\mathbf{B}\| \cos \theta$$

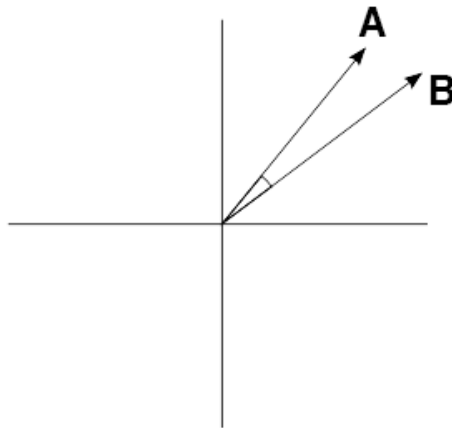


$$\text{Cosine distance} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$



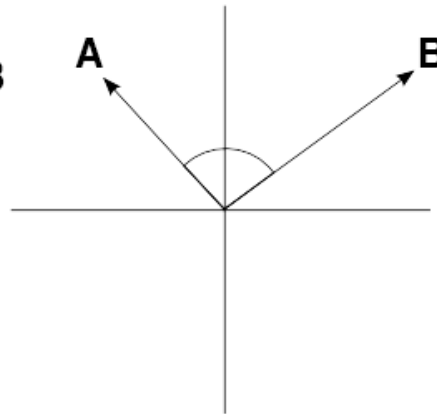
Cosine metrics idea...

Similar



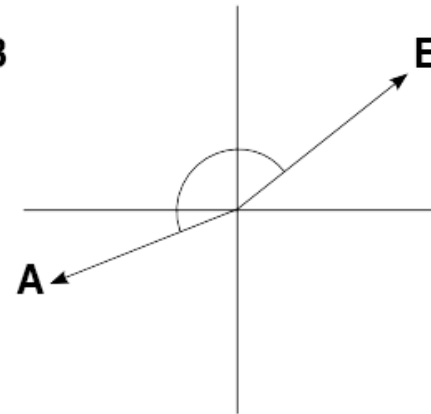
Dist. $\rightarrow 1$

Unrelated



Dist. $\rightarrow 0$

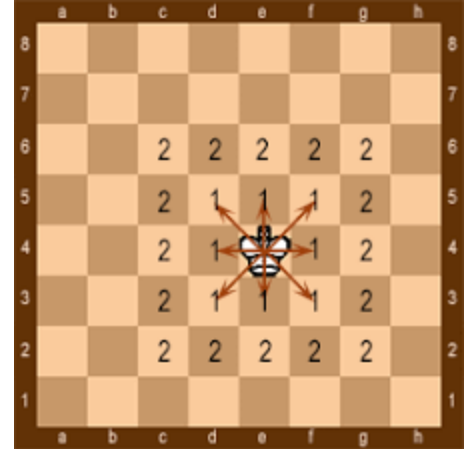
Opposite



Dist. $\rightarrow -1$

$$\text{Cosine distance} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

Chebyshev distance (L-infinity or chessboard)

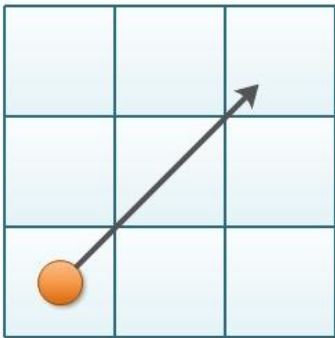


- Gives the **largest magnitude among each element of a vector**
- Ex.: having the vector $X = [-6, 4, 2]$, the L-infinity norm is 6 w.r.t. the origin
- In general, given two vectors p and q

$$d(p, q) = \max_i \{ |p_i - q_i| \}$$

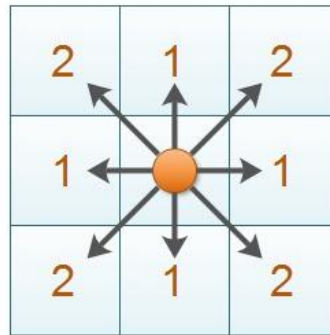
Comparison

Euclidean Distance



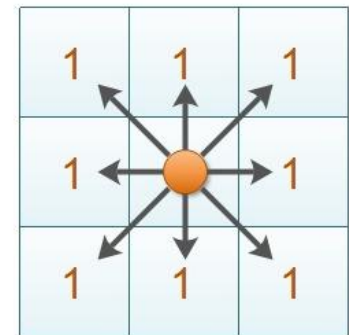
$$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Manhattan Distance



$$|x_1 - x_2| + |y_1 - y_2|$$

Chebyshev Distance



$$\max(|x_1 - x_2|, |y_1 - y_2|)$$

Minkowski distance

$X = (x_1, x_2, \dots, x_n)$ and $Y = (y_1, y_2, \dots, y_n) \in \mathbb{R}^n$

$$D(X, Y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}}$$

This metric
is **adimensional**

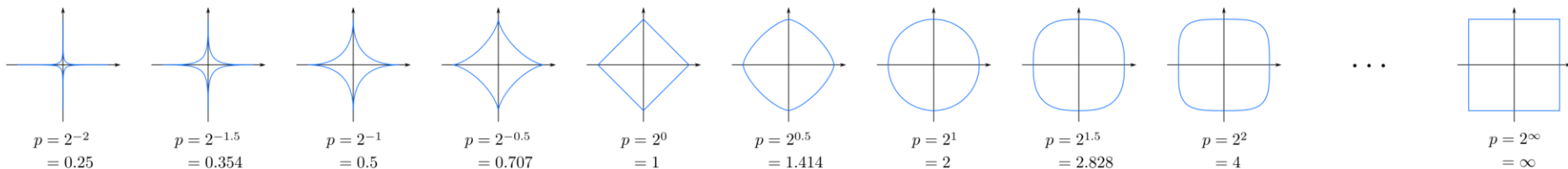
- $p = 1$ is the **Manhattan** distance.
Synonyms are L1-Norm, Taxicab or City-Block distance.
- $p = 2$ is the **Euclidean** distance.
Synonyms are L2-Norm or Ruler distance.
- $p \rightarrow \infty$ is the **Chebyshev** distance.
Synonyms are L-infinity or Lmax-Norm or Chessboard distance

Minkowski distance

$$D(X, Y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}}$$

Example with two dimensions → N=2

The set of all points that are at the unit distance from the centre (**unit circle**) with various values of p



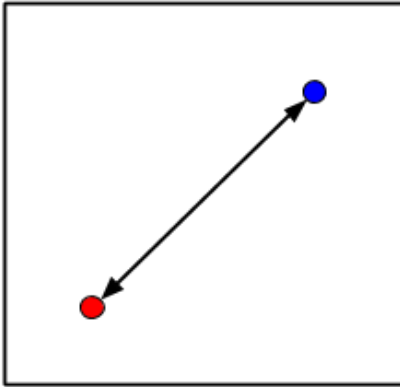
$p = 1$ is the **Manhattan** distance.
Synonyms are L1-Norm, Taxicab or City-Block distance.

$p = 2$ is the **Euclidean** distance.
Synonyms are L2-Norm or Ruler distance.

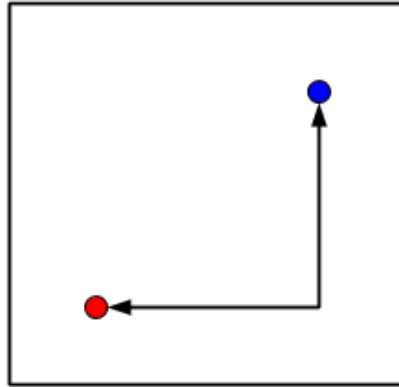
$p \rightarrow \infty$ is the **Chebyshev** distance.
Synonyms are L-infinity or Lmax-Norm or Chessboard distance

Overview of basic metrics

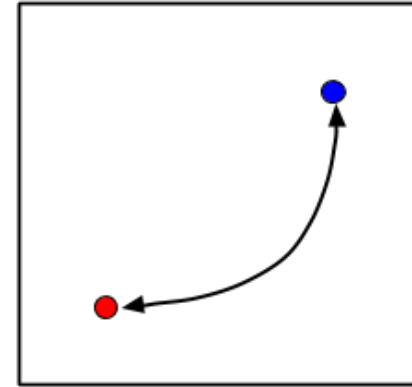
Euclidean



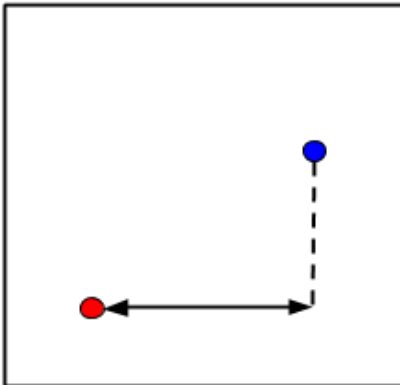
Manhattan



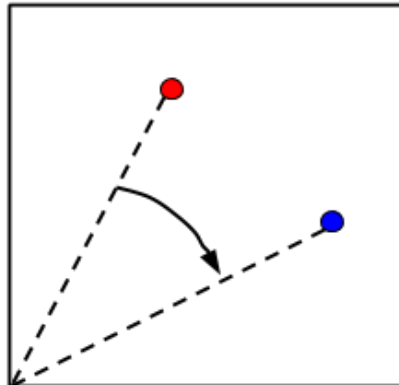
Minkowski



Chebychev



Cosine Similarity



Hamming





THEORY

Similarity in images

What is relevant what is useless?

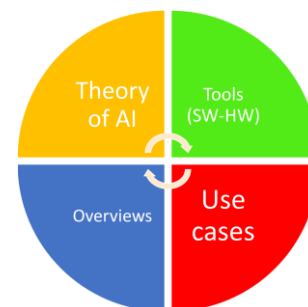
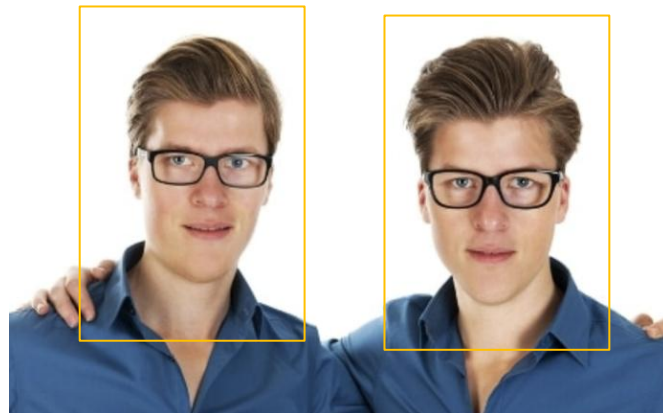


Image Similarity

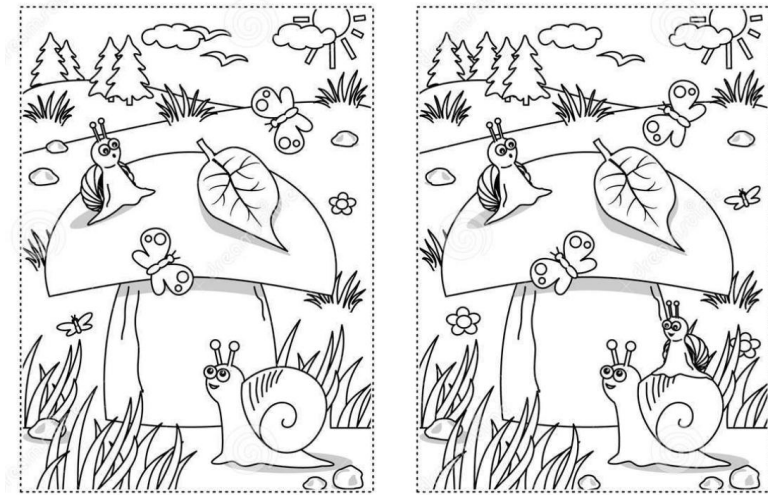


Image Similarity compares two images and returns a value that tells you how visually similar they are

`score = similarity(image1, image2)`

- If Score $\rightarrow 0$ so images $\rightarrow$ Contextually similar
- If (score == 0) so images are identical

Image Similarity:

3 main usages scenarios

- **Clean out Datasets**
 - Messy datasets are critical for the ML designer, and cleaning out duplicates is a painful process
- **Image Similarity Search**
 - Have a picture of something and want to see if you have visually similar images? Using Similarity find contextually similar matches in your media library or user data.
- **Track Changes in Imagery**
 - Sometimes it can be hard to see changes in a project you're working on or monitoring.

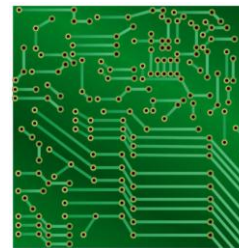
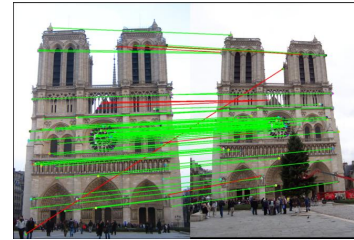


Image Similarity: 3 main approaches

- **Keypoint matching**

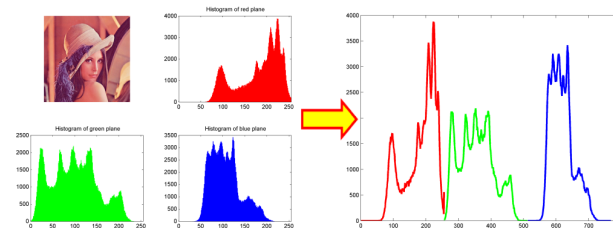
- SIFT
- SURF



- **Histogram**

- **Image hash**

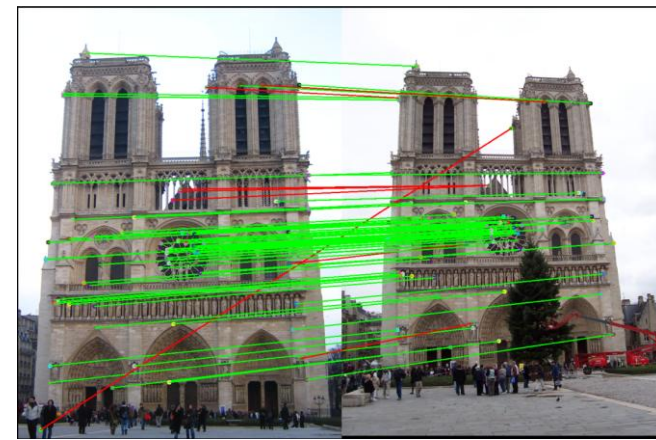
- aHash
- dHash
- pHash



0001100001010001100...1010100110001000000000



Image Similarity: Keypoint matching



- Procedure

1. Compute the abstraction of the image information and make a local decision at every image point to see if there is an image feature of the given type existing in that point.
2. Compare the detected feature between two images.

PROS

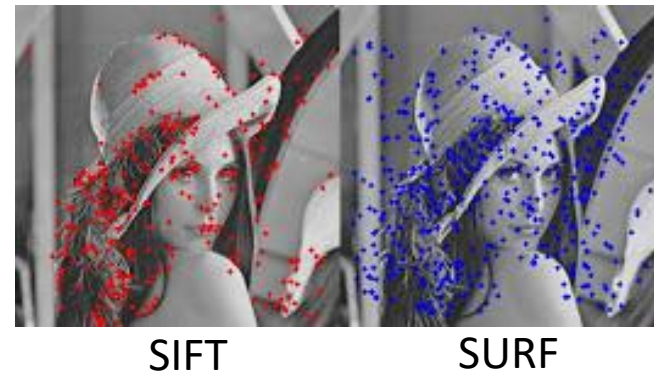
- These methods can match images under different scales, rotations, and lighting.

Very
important!

CONS

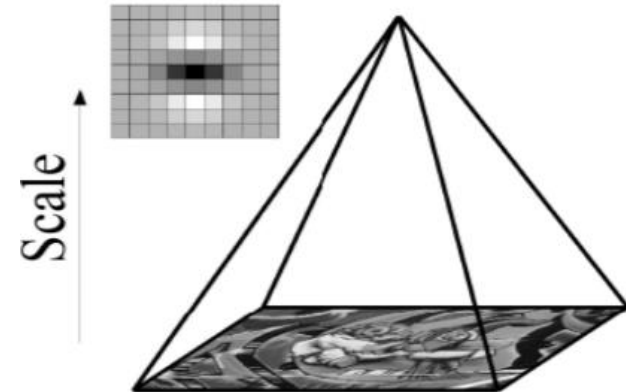
- running time of a naive implementation is $O(n^{2m})$,
 - n is the number of keypoints in each image,
 - m is the number of images in the database.

Keypoint matching: **SIFT and SURF**



- SCALE INVARIANT FEATURE TRANSFORM (SIFT)
- SPEEDED UP ROBUST FEATURE (SURF)
- SIFT/SURF quite powerful (many pub. libraries...)
 - Image similarity, object recognition, image registration, classification, 3D reconstruction, ...
- Discussion (in general not always):
 - **SURF** is better than **SIFT** in rotation invariant, blur and warp transform.
 - **SIFT** is better than **SURF** in different scale images.
 - **SURF** is faster (about 3x) than **SIFT**
 - **SIFT and SURF** are good in illumination changes images

Keypoint matching: SIFT

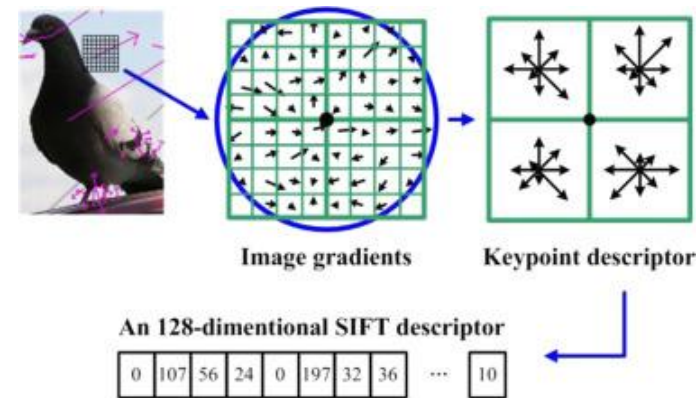


SCALE INVARIANT FEATURE TRANSFORM (SIFT)

1. Estimate a scale space extrema using the *Difference of Gaussian (DoG)*
2. A key point localization where the key point candidates are localized and refined by eliminating the low contrast points
3. A key point orientation assignment based on local image gradient and lastly a descriptor generator to compute the local image descriptor for each key point based on image gradient magnitude and orientation



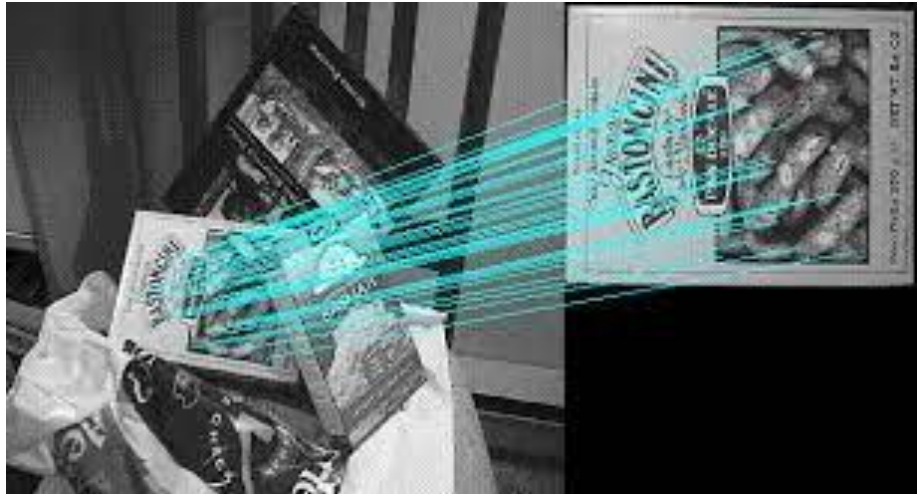
Keypoint matching: **SIFT** usage



- SIFT keypoints of objects are first extracted from a set of reference images and stored in a database.
- A similar image (or a large object inside) is recognized in a new image by individually comparing each feature from the new image to this database and finding candidate matching features based on Euclidean distance of their feature vectors.
- Plus other refining.. (Outside of the scope of the course)
- **SURF** is used with similar methods

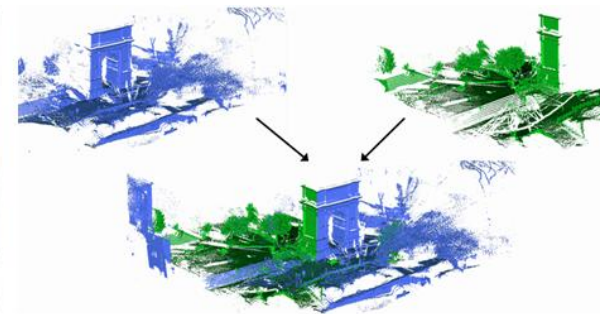
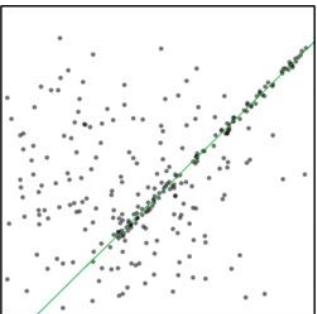
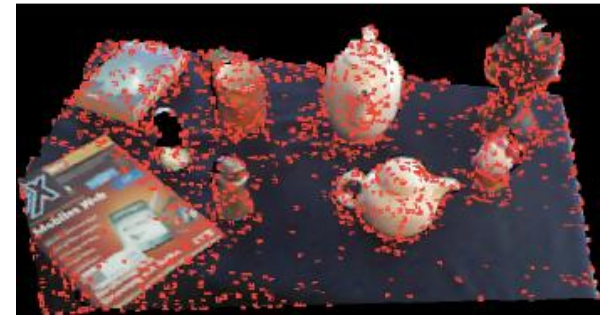
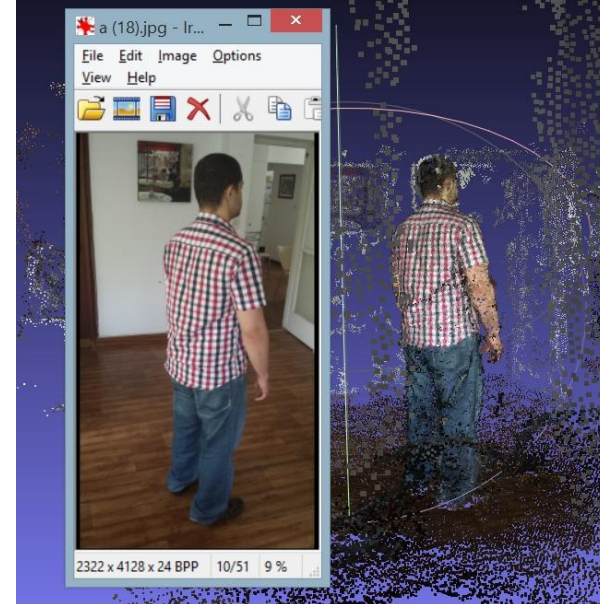
Example of usage of SIFT

- Find an object in a complex scene even with different scale and rotations
- The power of this technique is impressive



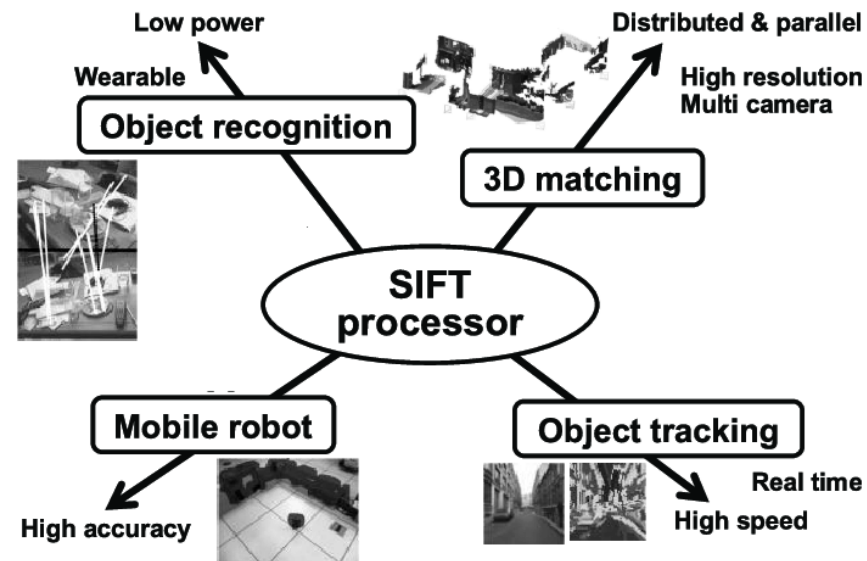
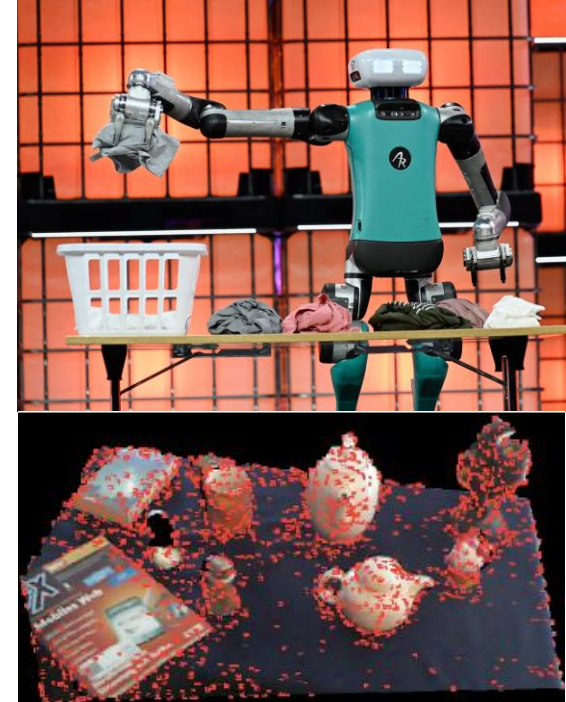
Example of usage of SIFT (2)

- Robust estimation is key in computer vision to get reliable results even when faced with **noisy or imperfect data**.
- By the identification of the same markers of the object in different views, make possible to **infer the 3D model** of the object and scenes.



Example of usage of SIFT (3)

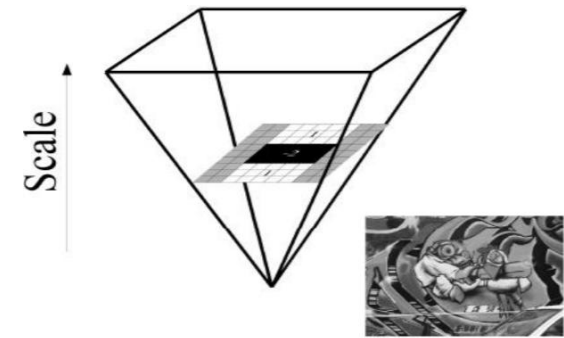
- Robots with cameras can reconstruct the scene in front of them during the motion
- SIFT is enabling many enhanced functionalities for Intelligent Systems!



Keypoint matching: SURF

- SPEEDED UP ROBUST FEATURE (SURF)

1. *SURF approximates the Difference of Gaussian DoG with box filters (Faster!)*
2. *BLOB detector which is based on the Hessian matrix to find the points of interest.*
3. *For feature description also SURF uses the wavelet responses.*



Different size of boxfilters
(Laplacian of Gaussian (LoG))
is convoluted with integral image.

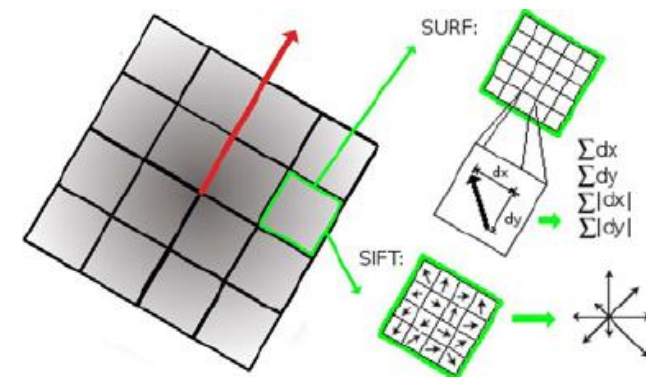
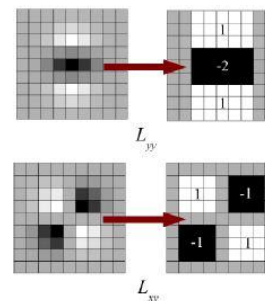
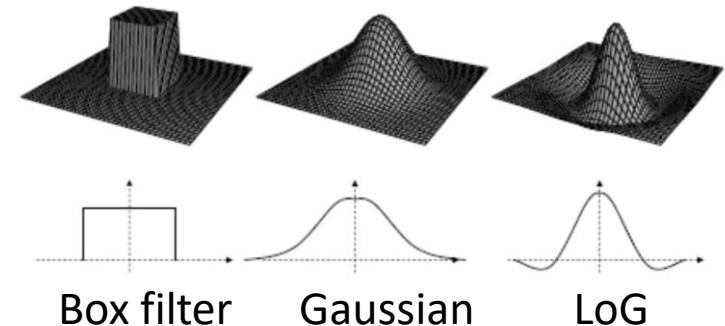


Image Similarity: Histogram comparison

- Procedure

1. Compute the histogram of the image (or quadrants, or blocks)
2. Create a feature vector
3. Use a metric to compare images (Wasserstein m.)

- PRO

- Very fast. Comparison is now on a short 1D vector
- Scale and rotation compliant

- CON

- **Shapes/Patterns are not relevant!**
- It is too simplistic.
A banana and a beach
will be similar

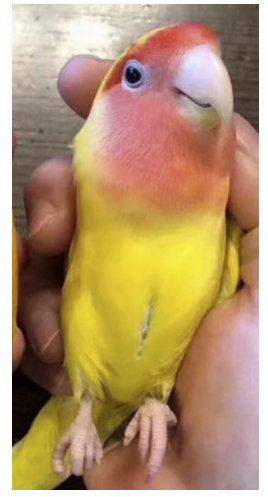
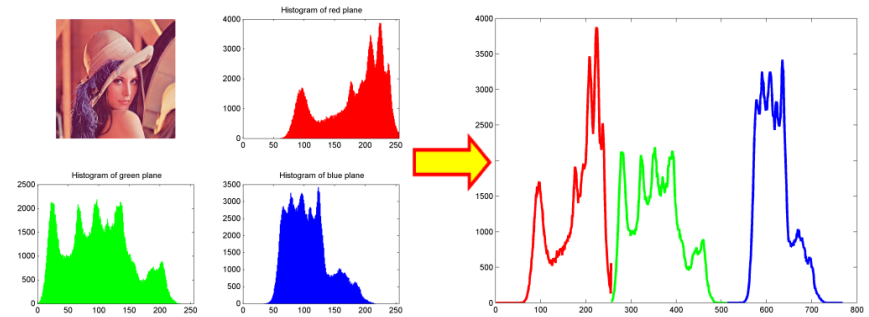
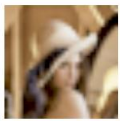


Image Similarity

Image hash

- The image is reduced down to a small hash code (like a “fingerprint”) by identifying salient features in the original image file and hashing a compact representation of those features (rather than hashing the image data directly).
- Finally, count the number of bit positions that are different (Hamming distance).

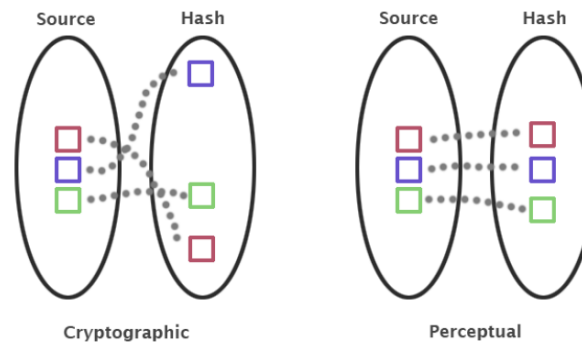
	Origin	Reduce size	Reduce color	Hash
X1				00011000010101001100...1010100110001000000000
X2				000110000101010001100...1010100110001000000000

A blue arrow points from the 15th bit position (the 15th '0' in the second hash) to the 15th bit position in the first hash, indicating a difference in the Hamming distance.

aHash, pHash, dHash

- **3 methods with similar initial steps**
- Looking at **a**verage, **p**erceptive and **g**radient difference and creating a binary feature
- **aHash** (a = Average)
 - 1) Reduce size (example: 8x8 R,G,B)
 - 2) Reduce to gray (8x8)
 - 3) Reduce to bits (0 if gray > below the mean gray)
 - 4) Merge the bits into a **64-bit integer**. ← your hash

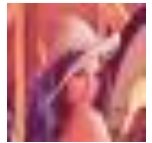
pHash



- **pHash** (p = Perceptive, works in the frequency domain)

- 1) Reduce size (32x32 R,G,B)
- 2) Reduce to gray (32x32)

32 x 32

$$\begin{bmatrix} 88 & 85 & \dots & 72 \\ 87 & 87 & \dots & 114 \\ \vdots & \vdots & \ddots & \vdots \\ 12 & 76 & \dots & 21 \end{bmatrix}$$


- 3) Compute the Discrete Cosine Transform (DCT).

It separates the image into a collection of frequencies and scalars (JPEG, and many other format...)

32 x 32

$$\begin{bmatrix} 8365 & -1554 & \dots & -17 \\ -254 & 338 & \dots & 13 \\ \vdots & \vdots & \ddots & \vdots \\ -424 & 74 & \dots & 1.4 \end{bmatrix}$$

- 4) Reduce the DCT (just keep the top-left 8x8)

- 5) Compute the mean DCT value T
(not the first value, but using the other 63 values)

8 x 8

$$\begin{bmatrix} 8365 & -1554 & \dots & -17 \\ -254 & 338 & \dots & 13 \\ \vdots & \vdots & \ddots & \vdots \\ -424 & 74 & \dots & 1.4 \end{bmatrix}$$

- 6) Reduce to bits (0 if DCT > below T)

- 7) Merge the bits into a **64-bit integer**. ← your hash

Images with same pHash

Cluster 802f2a2f2a7f2ad5



Cluster 80542a562f5f7f4a



Cluster 91b16ece313e34c9



Cluster a0008a0020002000



Cluster aa00800080008000



Cluster aa00800080008200



Cluster aa74f57e942d8225



Cluster ab2df43cd07285c3



= 8a0303f6df3ec8cd

```
https://pypi.org/project/ImageHash/  
>>> hash = imagehash.average_hash(Image.open('test.png'))  
>>> print(hash)  
d879f8f89b1bbf
```

dHash (very fast)

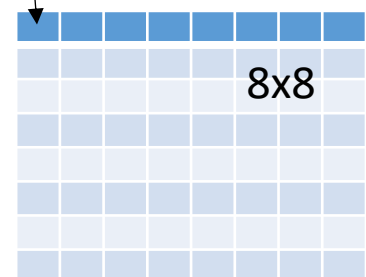
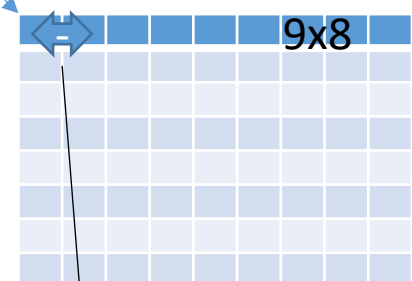
- **dHash** (d = Difference)

Calculate the difference for each of the pixel and compares the difference with the average differences

- 1) Reduce size (9x8 R,G,B)
- 2) Reduce to gray (72 gray pixels)
- 3) Compute the Differences

- Difference between adjacent pixels (relative gradient directions). The 9 pixels per row yields 8 differences between adjacent pixels. Eight rows of eight differences becomes 64 bits.
- Each bit is simply set based on whether the left pixel is brighter than the right pixel

- 4) Merge the bits into a **64-bit integer**. ← your hash



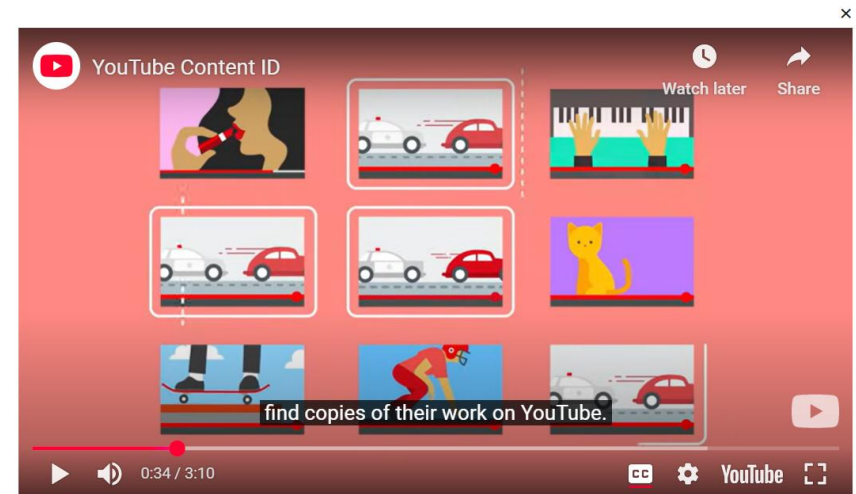
[10101...1]

Copyright checks and Video Fingerprinting

- Applying similar techniques (Content ID) YouTube is checking similarity in frames in video to check you uploaded some copyrighted frames in your video
- The exact method is not disclosure to avoid helping countermeasures
- In Video Fingerprinting similar technique to pHash and SIFT are applied to sound and frames groups to create an unique identifier

How Content ID works

Some copyright owners use Content ID, YouTube's automated content identification system, to easily identify and manage their copyright-protected content on YouTube.



Main points



- **Labelling errors**
 - Supervisor errors
 - Changes in time
 - Checks
- **Similarity and Distance**
 - The general framework of pattern recognition
 - Metrics in Datasets
 - Euclidean, Manhattan, Cosine, Chebyshev, Minkowski
 - Metrics Images
 - Histograms, SIFT, SURF
 - aHash, dHash, pHash